

Lecture 2 2026-09-03

Today: linear algebra "crash course",
gradient descent

Vector norms: assign "size" to a vector
that we will later want to minimize

properties: $\|x\|_p$ (p-norm)

$$1. \quad \|x\|_p \geq 0 \quad \forall x \in \mathbb{R}^n \\ \text{and } \|x\|_p = 0 \text{ if and only if } x = 0$$

$$2. \quad \|c x\|_p = |c| \|x\|_p \text{ for } c \in \mathbb{R}$$

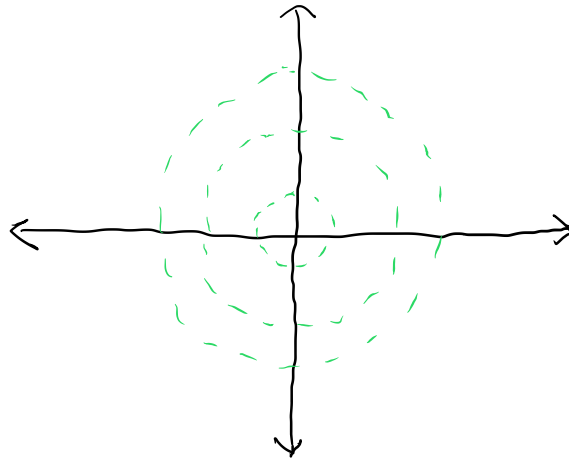
$$3. \quad \|x + y\|_p \leq \|x\|_p + \|y\|_p$$

$$p\text{-norm } \|x\|_p = \left(\sum_{i=1}^n |x_i|^p \right)^{1/p}$$

2-norm: $\|x\|_2 = \sqrt{\sum_{i=1}^n (x_i)^2}$

colloquially: "as the crow flies"

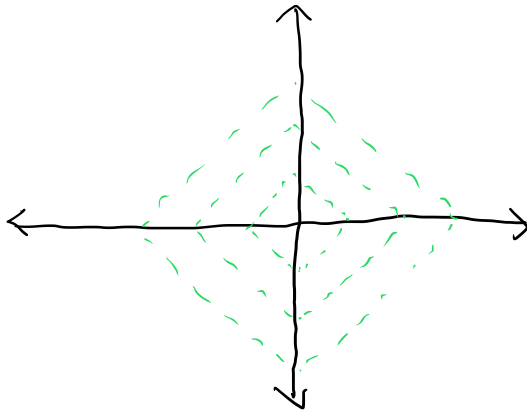
level sets look like:



1-norm: $\|x\|_1 = \sum_{i=1}^n |x_i|$

colloquially: Manhattan distance or
"taxi cab" norm

level curves look like:



l_1 -norm is known as "sparsity promoting"

i.e., minimize $\|x\|_1$

often yields more 0-elements in the optimal solution.

→ useful property in e.g., signal processing, ML

Property: if p, q are integers such that $0 < p < q$, then

$$\|x\|_p \geq \|x\|_q \text{ for all } x$$

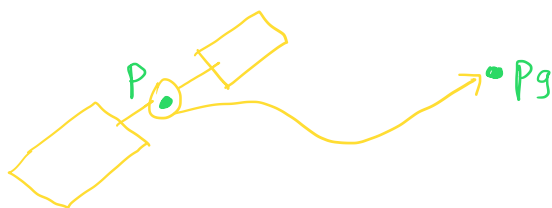
Why do we care?

Later on, we'll use the $\|x\|_p$ norm to minimize useful physical properties

e.g., if a spacecraft position is described as $p = (p_x, p_y) \in \mathbb{R}^2$, then

$$\underset{p}{\text{minimize}} \|p - p_g\|_p$$

will minimize position with respect to some desired goal p_g



Eigenvalues

If A is a square matrix

$A \in \mathbb{R}^{n \times n}$, then eigenvalue λ

$$Ax = \lambda v$$

Solve for eigenvalue via its
characteristic equation:

$$\det |A - \lambda I| = 0$$

Eigenvalue decomposition

Let $v_1, \dots, v_n$ be n linearly independent eigenvectors of A .

Then we can express $A = T \Lambda T^{-1}$

$$\text{where } T = \begin{pmatrix} v_1 & v_2 & \dots & v_n \end{pmatrix}$$

$$\Lambda = \begin{pmatrix} \lambda_1 & & & 0 \\ & \lambda_2 & & \\ & & \ddots & \\ 0 & & & \lambda_n \end{pmatrix}$$

Computational complexity

How does the runtime of an algorithm scale with respect to the size n with respect to its argument/input?

→ The choice and performance of the optimization algorithm we will use is determined by computational complexity

Common to consider big- O notation:

$O(\cdot)$ input size n

examples:

$O(n)$: linear time

adding two vectors of size n

```
def add(x, y):  
    sum = (0, ..., 0)  
    for i = 1 → n  
        sumi = xi + yi  
    return sum
```

matrix multiplication $y = Ax$ $A \in \mathbb{R}^{m \times n}$

$$\begin{bmatrix} a_1 \\ \vdots \\ a_m \end{bmatrix} \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}$$

def multiply(A, x):

$y = (0, \dots, 0)$

 for $i = 1 \rightarrow m$

$y_i = 0$

 for $j = 1 \rightarrow n$

$y_i += A_{ij} x_j$

 return y

Matrix inverse: A is square and invertible

$$A \in \mathbb{R}^{n \times n}$$

→ complexity of A^{-1} is $\mathcal{O}(n^3)$

→ "optimized" routines used in high-performance

linear algebra libraries get down to

$$\mathcal{O}(n^w) \text{ where } w \approx 2.40$$

Gradient descent

We will study problems of the form:

$$\min_{x \in \mathbb{R}^n} f(x)$$

$$\text{subj. to: } f_i(x) \leq 0 \quad i = 1, \dots, n_{\text{ineq}}$$

$$h_i(x) = 0 \quad i = 1, \dots, n_{\text{eq}}$$

Gradient:

if $f(x)$ such that $f: \mathbb{R}^n \rightarrow \mathbb{R}$

and $f \in \mathcal{C}^1$

$$\rightarrow \text{gradient } \nabla f(x) = \begin{pmatrix} \partial f / \partial x_1 \\ \vdots \\ \partial f / \partial x_n \end{pmatrix}$$

Hessian:

Let $f: \mathbb{R}^n \rightarrow \mathbb{R}$ and $f \in \mathcal{C}^2$

$$\text{then Hessian } \nabla_{xx} f(x) = \begin{pmatrix} \partial^2 f / \partial x_1^2 & \dots & \partial^2 f / \partial x_1 \partial x_n \\ \vdots & \ddots & \vdots \\ \partial^2 f / \partial x_n \partial x_1 & \dots & \partial^2 f / \partial x_n^2 \end{pmatrix}$$

H is a symmetric matrix and $H \in \mathbb{R}^{n \times n}$

The gradients and Hessians will be useful when we want to construct a local approximation of $f(x)$ using Taylor series

recall scalar version when $f: \mathbb{R} \rightarrow \mathbb{R}$

$$\rightarrow f(x) \approx f(\bar{x}) + f'(\bar{x})(x - \bar{x}) + \frac{1}{2} f''(\bar{x})(x - \bar{x})^2$$

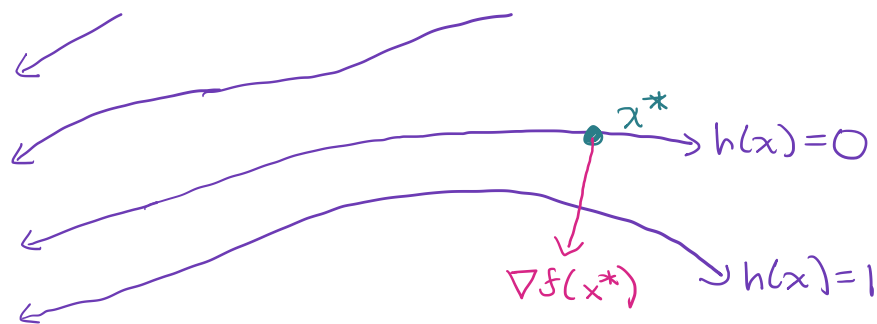
in vector version for $f: \mathbb{R}^n \rightarrow \mathbb{R}$

$$\begin{aligned} \rightarrow f(x) \approx f(\bar{x}) + \nabla f(\bar{x})^T (x - \bar{x}) \\ + \frac{1}{2} (x - \bar{x})^T \nabla_{xx} f(\bar{x}) (x - \bar{x}) \end{aligned}$$

What information about a function does the gradient $\nabla f(x)$ provide?

Consider the level curves of some function $f(x)$.





A gradient points **orthogonal** to the level curve and in the direction of increasing value (low-to-high)

Key insight: if we want to minimize $f(x)$,
then $\nabla f(x)$ can provide us information about
direction of increase (and hence decrease)